

Self-Correcting RAG, Interactively: Teaching Reflective Retrieval (Self-RAG / CRAG) by Contrast with the Naive Pipeline It Fixes

Chandra Prakash Bathula · chandu.bathula1617@gmail.com · NeurIPS 2026 Educational Resources Track · Interactive web application + exercises (full source included)

1 The Concept: Reflective, Self-Correcting Retrieval

The emerging concept this resource teaches is **self-corrective retrieval-augmented generation**: pipelines in which the model *critiques its own retrieval and generation* — deciding whether to retrieve at all, grading retrieved evidence for relevance, verifying that a drafted answer is supported by that evidence, and rewriting the query and retrying when a check fails. Introduced by Self-RAG's reflection tokens [1] and CRAG's retrieval evaluator with corrective query rewriting [2], with adaptive retrieval policies from FLARE [3], this reflect-verify-retry loop has become a default ingredient of current agentic search systems: 2025–2026 work trains exactly these routing/verification behaviors with reinforcement learning (e.g. Search-R1 [4]) and surveys them as the "agentic RAG" design space [5]. It is the kind of technique researchers reach for right now, and it is invisible inside frameworks — which is why learners need to see it run.

What learners do. The resource is an interactive web application. Learners first assemble a working naive RAG pipeline over any document they upload (extract → chunk → embed → retrieve → generate) — deliberately as *baseline scaffolding*, so they can feel naive RAG's core failure mode: it trusts whatever the top-k chunks are and answers once, unverified. Then they flip one toggle and run the same question through the self-correcting loop, with every decision rendered live: the routing verdict, per-chunk relevance grades, the draft, the support verdict, query reformulations, and per-call token/latency costs. A side-by-side compare mode makes the concept's value and its price concrete. All grading is *fail-closed*: when a verification step cannot be parsed, the UI reports an unverified, low-confidence answer — the resource models honest verification rather than hiding failure.

2 Learning Objectives

LO1. Trace a self-corrective RAG loop — routing, evidence grading, answer verification, query reformulation — and map each step to its origin in Self-RAG [1] and CRAG [2] (the trace UI annotates every step with this mapping).

LO2. Explain the accuracy/cost trade-off of reflection using the measured call counts, tokens, and latencies the tool displays, and steer it with the exposed knobs (top-k, max correction passes, relevance threshold).

LO3. Identify naive RAG's failure modes — irrelevant retrieval trusted unconditionally, unsupported claims, reformulation drift — and observe which ones the self-correcting loop catches, and why verification should fail closed.

LO4. (Scaffolding) Explain each baseline pipeline stage: how chunking bounds retrievable evidence, how cosine similarity over normalized embeddings ranks it, and why low-dimensional projections of embedding space are suggestive rather than faithful.

LO5. Contrast plain-LLM, naive-RAG, and self-corrective answers on the same question and evidence.

3 Prerequisites and Difficulty Levels

Prerequisites. Comfort using a web application; a general idea of what an LLM is. Vectors/similarity help but are introduced visually. No programming needed to use the tool.

Level A (pre-university/intro). Baseline pipeline with defaults; plain-LLM vs RAG contrast (LO4, LO5). **Level B (undergraduate).** Vary chunking and embedding models; read similarity scores and projections critically (LO4).

Level C (graduate/practitioner). The concept itself: agentic mode with the trace open, threshold/loop experiments, cost analysis, paper mapping (LO1–LO3). Guided exercises with solutions are included for each level, grounded in the bundled open-license example corpus (a 1,200-page nutrition textbook).

4 Materials and Accessibility

The submission contains the full open-source application (MIT; React + FastAPI), setup for fully local use (Ollama + open embedding models; no paid APIs required), the graded exercise set with solutions, and the example corpus. A CI-tested suite of 40+ unit/integration tests accompanies the code. All interactive controls are keyboard-accessible with ARIA roles; visualizations ship a table-view alternative and captions stating their limits. A transparent "Demo Mode" (persistently bannered in the UI and disclosed here) stages one failed retrieval pass so instructors can reliably demonstrate the correction loop live; all other steps run genuinely. All materials are

original and were created for this track.

References

- [1] A. Asai, Z. Wu, Y. Wang, A. Sil, H. Hajishirzi. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. *ICLR*, 2024. arXiv:2310.11511.
- [2] S.-Q. Yan, J.-C. Gu, Y. Zhu, Z.-H. Ling. Corrective Retrieval Augmented Generation. arXiv:2401.15884, 2024.
- [3] Z. Jiang et al. Active Retrieval Augmented Generation. *EMNLP*, 2023. arXiv:2305.06983.
- [4] B. Jin et al. Search-R1: Training LLMs to Reason and Leverage Search Engines with Reinforcement Learning. arXiv:2503.09516, 2025.
- [5] A. Singh et al. Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG. arXiv:2501.09136, 2025.
- [6] P. Lewis et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS*, 2020 (the established baseline this resource uses as scaffolding).